Exploratory Data Analysis

Introduction

This notebook presents the Exploratory Data Analysis (EDA) for the **PlaySafe** project.

The purpose of this analysis is to obtain a basic understanding of the cleaned LaLiga player dataset before developing the first machine-learning prototype in Iteration Zero.

The analysis focuses on the main characteristics of the dataset, the distribution of relevant variables, and the remaining missing values. The EDA is intentionally kept concise because Iteration Zero is designed as a quick and simple proof of concept using k-Nearest Neighbors (k-NN).

The findings from this analysis will be used to support the preparation of the dataset for the first model.

Dataset Overview

The first step of the analysis is to inspect the cleaned dataset and understand its general structure. This includes checking the number of rows and columns, reviewing the available variables and examining a small sample of the data.

This initial inspection provides an overview of the dataset before analysing the individual variables and their relationships with injury information.

In [10]:
```
import pandas as pd
import numpy as np

# Load the cleaned dataset
df = pd.read_csv("../data/processed/laliga_cleaned.csv")

print("Shape:", df.shape)
df.head()
```

```
Shape: (3713, 14)
```

Out[10]:

	player_name	date_of_birth_age	height	position	current_age_x	minutes_played
0	Thibaut Courtois	May 11 1992 (32)	200.0	Goalkeeper	32	2970.0
1	Thibaut Courtois	May 11 1992 (32)	200.0	Goalkeeper	32	2970.0
2	Thibaut Courtois	May 11 1992 (32)	200.0	Goalkeeper	32	513.0
3	Thibaut Courtois	May 11 1992 (32)	200.0	Goalkeeper	32	513.0
4	Thibaut Courtois	May 11 1992 (32)	200.0	Goalkeeper	32	4920.0

Before analysing individual variables, the structure of the cleaned dataset is inspected. This includes the number of records, the number of unique players, and the number of records associated with each player.

This is important because the dataset can contain multiple records for the same player, as a player may have multiple injury records across different seasons.

In [11]:
```
print("Number of rows:", len(df))
print("Number of columns:", len(df.columns))
print("Number of unique players:", df["player_name"].nunique())
```

```
Number of rows: 3713
Number of columns: 14
Number of unique players: 490
```

In [12]:
```
df["player_name"].value_counts().head(10)
```

Out[12]:
```
player_name
Juan Cruz                40
David Alaba              22
Andreas Christensen      22
JosÃ© MarÃa GimÃ©nez     22
William Carvalho         22
Antonio RÃ¼diger          21
Viktor Tsygankov         21
Daniel Carvajal          20
Ricardo RodrÃguez       20
Giovani Lo Celso         20
Name: count, dtype: int64
```

In [14]:
```
player_season_counts = (
    df.groupby(["player_name", "season"])
      .size()
      .sort_values(ascending=False)
)

player_season_counts.head(10)
```

Out[14]:
```
player_name       season
Juan Cruz         22/23     8
                  23/24     8
David Alaba       21/22     8
Antonio RÃ¼diger  18/19     6
Marc Bartra       16/17     6
Viktor Tsygankov  20/21     6
Nico Williams     23/24     6
Arsen Zakharyan   23/24     5
Marcos Llorente   21/22     5
Matija NastasiÄ‡  17/18     5
dtype: int64
```

The results show that multiple records can exist for the same player and season. This indicates that the dataset is based on individual records rather than one record per player-season. Therefore, repeated player records should not be removed as duplicates at this stage

The data types and number of non-null values are inspected to identify the structure of the variables and the presence of missing values

In [15]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 3713 entries, 0 to 3712
Data columns (total 14 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   player_name        3713 non-null   str
 1   date_of_birth_age  3713 non-null   str
 2   height             3696 non-null   float64
 3   position           3713 non-null   str
 4   current_age_x      3713 non-null   int64
 5   minutes_played     3704 non-null   float64
 6   season             3704 non-null   str
 7   injury             1776 non-null   str
 8   from_date          1776 non-null   str
 9   to_date            1774 non-null   str
 10  days_out           1776 non-null   float64
 11  matches_missed     1676 non-null   float64
 12  injury_year        1776 non-null   float64
 13  age_at_injury      1776 non-null   float64
dtypes: float64(6), int64(1), str(7)
memory usage: 406.2 KB
```

The main numerical variables are summarised using descriptive statistics. This provides an initial overview of their ranges, central values, and variation within the dataset.

In [16]:
```
df.describe()
```

Out[16]:

	height	current_age_x	minutes_played	days_out	matches_missed	injur
count	3696.000000	3713.000000	3704.000000	1776.000000	1676.000000	1776.(
mean	182.073593	27.733639	2670.028348	35.085586	6.026253	2020.7
std	6.520292	4.220663	1539.834739	49.140276	7.960381	2.(
min	164.000000	17.000000	0.000000	1.000000	1.000000	2012.(
25%	177.000000	25.000000	1485.000000	9.000000	2.000000	2019.(
50%	182.000000	28.000000	2603.500000	18.000000	3.000000	2021.(
75%	187.000000	31.000000	3621.750000	38.000000	7.000000	2023.(
max	200.000000	40.000000	9003.000000	381.000000	64.000000	2025.(

The number of missing values in each variable is examined to identify where missing data is concentrated in the dataset.

In [17]:
```
df.isna().sum().sort_values(ascending=False)
```

Out[17]:
```
matches_missed       2037
to_date              1939
injury               1937
from_date            1937
days_out             1937
injury_year          1937
age_at_injury        1937
height                 17
minutes_played          9
season                  9
current_age_x           0
position                0
player_name             0
date_of_birth_age       0
dtype: int64
```

The missing values are mainly concentrated in the injury-related variables. The variables injury, from_date, days_out, injury_year, and age_at_injury each have 1,937 missing values, while to_date has 1,939 and matches_missed has 2,037. Only a small number of values are missing for height, minutes_played, and season.

The main categorical variables, position and season, are examined to understand their distribution within the dataset.

In [18]:
```
print("Positions:")
print(df["position"].value_counts())

print("\nSeasons:")
print(df["season"].value_counts().sort_index())
```

```
Positions:
position
Centre-Back           731
Central Midfield      451
Centre-Forward        423
Goalkeeper            360
Right-Back            346
Right Winger          305
Left Winger           302
Defensive Midfield    278
Left-Back             268
Attacking Midfield    208
Right Midfield         25
Second Striker         16
Name: count, dtype: int64

Seasons:
season
12/13     84
13/14    119
14/15    158
15/16    185
16/17    227
17/18    291
18/19    331
19/20    319
20/21    438
21/22    444
22/23    427
23/24    431
24/25    250
Name: count, dtype: int64
```

The dataset contains 12 different player positions. Centre-Back is the most represented position, followed by Central Midfield and Centre-Forward.

The dataset covers 13 seasons, from 12/13 to 24/25. The number of records differs between seasons, with the highest number of records in the 20/21 and 21/22 seasons.

These differences in the number of records across positions and seasons should be considered when preparing the data